

Security Architecture & Technical Controls

BCP Co-Curricular Management System (SMS2) — Comprehensive Security Reference

VERIFIED SECURE

System ID: SMS2-COCURRICULAR
Target Platform: PHP 8.2 / MySQL 8.0 / Apache
Classification: Institutional Technical Documentation

Executive Security Overview

The **Bestlink College of the Philippines (BCP) Co-Curricular Management System** employs a defense-in-depth security model to govern student organizations, institutional finance, campus-wide elections, and event tracking. The application implements rigorous role-based segregation, multi-layer cryptographic safeguards, automated state validation, and persistent forensic logging to ensure compliance with the *Philippine Data Privacy Act of 2012 (RA 10173)* and institutional IT security policies.

I. Architectural Security Controls & Technical Definitions

11 Core Feature Modules

1. Centralized Authentication & Session Hardening

Identity & Access

FORMAL DEFINITION

The verification of user identity through credential validation, coupled with stateful session controls that prevent session hijacking, fixation, and unauthenticated access.

- Enforced Access Gate:** The `require_auth()` guard verifies that an authenticated `user_id` exists in the session. Unauthenticated web requests are redirected to `signin.php`; unauthenticated AJAX/fetch requests are terminated with an HTTP `401 Unauthorized` JSON payload.
- Session Fixation Neutralization:** Upon successful login, `session_regenerate_id(true)` invalidates the pre-login session ID and creates a fresh cryptographic token, preventing session fixation exploits.
- Cookie Hardening Directives:** Sessions are configured with `HttpOnly=1` (blocks JavaScript cookie access), `SameSite=Lax` (prevents cross-site cookie leaks), `use_only_cookies=1`, and conditional `Secure=1` over TLS.
- Disabled Public Registration:** Public self-registration is permanently disabled in the backend handler to ensure zero unvetted accounts enter the directory.

Source Files: `app/shared/security.php` (L75-86), `app/shared/auth_actions.php` (L27-68)

2. Role-Based Access Control (RBAC) & Least Privilege

Authorization

FORMAL DEFINITION

An access management model restricting system actions and data scopes based on assigned user roles, ensuring each actor can only perform actions strictly necessary for their function.

- Hierarchical Role Architecture:** Defines four distinct operational personas: `student`, `club_adviser`, `ssc` (Supreme Student Council), and `admin` (System Administrator).
- Guard Middleware:** `require_role(array $allowed)` terminates unauthorized requests with HTTP `403 Forbidden`.
- Absolute Financial Lockout:** General students are strictly barred from financial workflows. Direct visits to `budget.php` or API calls to `budget_actions.php` are rejected immediately.
- Adviser Club Scoping:** Faculty advisers are scoped strictly to their assigned organization ID (`club_id`) for approvals, elections, rosters, and disbursements.
- Server-Side DOM Gating:** In `admin_system.php`, user management, matrix permissions, and maintenance operations tabs are omitted from the server response for non-admins, locking SSC officers strictly to read-only audit views.

Source Files: `app/shared/security.php` (L91-104), `app/shared/budget_actions.php` (L23-26), `app/dashboard/admin_system.php`

3. Privilege Escalation Prevention & Persona Auditing

Integrity

FORMAL DEFINITION

Defensive controls preventing lower-privileged users from escalating their authorization level through parameter tampering, session manipulation, or request spoofing.

- **Real-Role Verification:** The role-switching mechanism in `dashboard.php` checks `$_SESSION['real_role'] === 'admin'`. Non-administrators who append `?switch_role=admin` are denied access.
- **Tamper Logging:** Any unauthorized role-switch attempt is caught and written to the persistent audit log under `unauthorized_role_switch_attempt`, including IP and actor ID.
- **Impersonation Transparency:** When an administrator adopts a secondary role for testing, an amber banner is rendered with a secure revert link to `dashboard.php?switch_role=admin`.

Source Files: `app/dashboard/dashboard.php` (L28-39), `app/shared/sidebar.php` (L48-55)

4. Cross-Site Request Forgery (CSRF) Protection

Threat Defense

FORMAL DEFINITION

A security protocol that prevents malicious websites from forging unauthorized HTTP requests on behalf of an authenticated user.

- **Cryptographic Random Tokens:** Tokens are generated via `bin2hex(random_bytes(32))`, yielding 64 characters of high-entropy randomness per session.
- **Dual-Vector Ingestion:** `verify_csrf()` validates tokens from standard form POST bodies, JSON payloads, or HTTP headers (`X-CSRF-Token`).
- **Timing-Attack Immunity:** Token equality is evaluated using PHP's constant-time `hash_equals()` function, eliminating side-channel timing leaks.
- **Universal Enforcement:** Every state-mutating POST endpoint across the entire system executes CSRF verification as its first operational step.

Source Files: `app/shared/security.php` (L19-70), `app/shared/*_actions.php`

5. SQL Injection Prevention & Parameterized Queries

Database Security

FORMAL DEFINITION

Database query architecture ensuring user inputs are treated exclusively as literal data values rather than executable commands, neutralizing code injection.

- **Strict Prepared Statements:** All dynamic SQL queries utilize `mysqli::prepare()` with typed placeholders (`bind_param('s', ...)`, `'i'`, `'d'`).
- **No Concatenated Input:** User-supplied strings are never directly concatenated into SQL statements across authentication, student records, finance, or election logic.
- **UTF-8 Character Set Enforcement:** Connections enforce `utf8mb4` encoding, protecting against multibyte character injection and truncation vulnerabilities.

Source Files: `app/shared/db.php` (L73), `app/shared/admin_actions.php`, `app/shared/auth_actions.php`

6. Cross-Site Scripting (XSS) Defense & Security Headers

Web Hardening

FORMAL DEFINITION

Input sanitization, output encoding, and HTTP response headers that stop malicious scripts from executing in client browsers and mitigate UI redressing.

- **Contextual Output Escaping:** All dynamic user text rendered to the browser passes through `htmlspecialchars($val, ENT_QUOTES, 'UTF-8')` or the `e()` helper function.
- **SecurityHeadersMiddleware:** Enforces critical HTTP security headers on all responses:
 - `X-Content-Type-Options: nosniff` — Blocks MIME-type sniffing exploits.
 - `X-Frame-Options: SAMEORIGIN` — Prevents Clickjacking by disallowing unauthorized external framing.
 - `X-XSS-Protection: 1; mode=block` — Triggers browser-level XSS filtering.
 - `Referrer-Policy: strict-origin-when-cross-origin` — Restricts sensitive path leaks in referrers.

Source Files: `app/Http/Middleware/SecurityHeadersMiddleware.php`, `app/shared/security.php` (L109-111)

7. Cryptographic Password Hashing & Salt Protection

Cryptography

FORMAL DEFINITION

A one-way cryptographic transformation that converts plain text credentials into salted, irreversible hashes, rendering stored passwords unreadable.

- **Bcrypt Algorithm:** Passwords are hashed using PHP's `password_hash($pass, PASSWORD_DEFAULT)`, incorporating per-user cryptographically random salts and adjustable computational cost factors.
- **Safe Verification:** Logins verify passwords via `password_verify()`, eliminating timing attack vulnerabilities.
- **Zero Plaintext Storage:** Passwords are never stored or displayed in plaintext; administrators can trigger password resets to new hashes but cannot read existing credentials.

Source Files: `app/shared/auth_actions.php` (L47, L243), `app/shared/admin_actions.php` (L126)

8. Secure File Upload & Media Inspection

Storage Security

FORMAL DEFINITION

Server-side file inspection and sanitization controls preventing malicious executables (web shells) or oversized payloads from compromising the filesystem.

- **Binary Inspection:** Avatar uploads are verified using `getimagesize()` to inspect actual image magic bytes, rather than trusting client extensions or headers.
- **Strict MIME Whitelist:** Only `image/jpeg`, `image/png`, `image/webp`, and `image/gif` are permitted.
- **Payload Size Limits:** File sizes are capped at 5MB, preventing denial-of-service via storage exhaustion.
- **Cryptographic Renaming:** Files are renamed using `'avatar_' . $userId . '_' . time() . '_' . bin2hex(random_bytes(4)) . '.' . $ext`, defeating directory traversal and double-extension exploits (e.g., `shell.php.jpg`).
- **Traversal-Safe Deletion:** Prior avatar cleanup wraps filenames in `basename()` to block path traversal outside the designated storage path.

Source Files: `app/shared/auth_actions.php` (L70-154)

9. Security Audit Logging & Compliance Ledger

Forensics & Governance

FORMAL DEFINITION

An immutable, structured chronological record of security actions, user operations, and anomalies designed for forensic investigation and regulatory auditability.

- **Structured Metadata Capture:** `log_audit()` records the authenticated actor ID, categorized action slug, target database table, record ID, detailed narrative, and client IP address (`$_SERVER['REMOTE_ADDR']`).
- **Audited Milestones:** Captures critical events including credential resets, role alterations, unauthorized permission attempts, event approvals, attendance scans, and system settings modifications.
- **Compliance CSV Export:** System administrators can export filtered audit trails directly to encrypted CSV files for governance reporting.

Source Files: `app/shared/notification_actions.php` (L26-32), `app/shared/admin_actions.php` (L340-380)

10. Application-Specific Anti-Fraud & Data Integrity Controls

Domain Logic

FORMAL DEFINITION

Domain-specific validation rules and concurrency controls that prevent ballot stuffing, attendance forging, and unauthorized financial disbursement.

- **Election One-Vote Integrity:** When casting election ballots in `election_actions.php`, the system enforces:
 - Membership validation: Only verified active club members (`status = 'Active'`) can vote in club elections.
 - Adviser voting ban: Advisers supervise elections but are barred from student voting.
 - Concurrency locking: Uses `begin_transaction()` and row-level locking (`SELECT ... FOR UPDATE`) on `election_votes` to prevent double-voting under high concurrency.
- **Dual-Mode QR Attendance Anti-Spoofing:** Verifies student and event tokens, validates active status, rejects duplicate check-ins, logs the entry method (`QR` vs `manual`), and records audit logs for every scan.
- **Multi-Tier Financial Approval Pipeline:** Implements strict separation of financial duties: `Pending Adviser` (Endorsement) → `Pending SSC` (Vetting) → `Pending Admin` (Final Approval & Disbursement).

Source Files: `app/shared/election_actions.php` (L234-300), `app/shared/attendance_actions.php` (L165-205), `app/shared/budget_actions.php`

11. Installation Lock & Environment Credential Isolation

Infrastructure Hardening

FORMAL DEFINITION

Deployment safeguards that prevent database re-initialization and decouple sensitive credentials from application source code.

- **Installer Lockout:** Upon deployment, an `install.lock` file is created. Any subsequent request to execute `setup.php` or re-run database schema migrations is blocked with HTTP `403 Forbidden` unless authenticated as an active System Administrator.
- **Environment Variable Decoupling:** Database credentials (`DB_HOST`, `DB_USER`, `DB_PASS`, `DB_NAME`) are externalized in `.env` and parsed dynamically via `db.php`.
- **Repository Exclusion:** Sensitive environment files and local storage directories are excluded from source control via `.gitignore` and container packaging via `.dockerignore`.

Source Files: `app/shared/install.lock`, `app/shared/setup.php` (L15-62), `app/shared/db.php` (L6-34)

II. Threat Mitigation & Security Control Matrix

Mapping & Standards

Threat / Attack Vector	Mitigating Security Feature	Technical Defense Mechanism	Code Implementation
Session Hijacking / XSS Theft	Session Cookie Hardening	HttpOnly, SameSite=Lax, Secure flags prevent JavaScript reading session cookies	app/shared/security.php
Session Fixation	Session Regeneration	session_regenerate_id(true) on login invalidates attacker-known session ID	app/shared/auth_actions.php
Privilege Escalation	Real Role & Impersonation Gate	Strict verification of real_role === 'admin'; unauthorized attempts logged	app/dashboard/dashboard.php
Cross-Site Request Forgery	CSRF Token Validation	Dual-channel 64-char token check (POST & header) via constant-time hash_equals()	app/shared/security.php
SQL Injection (SQLi)	Parameterized Prepared Statements	mysqli::prepare() and typed bind_param() across all queries	app/shared/*_actions.php
Cross-Site Scripting (XSS)	Output Encoding & Headers	htmlspecialchars() + X-XSS-Protection & nosniff headers	SecurityHeadersMiddleware.php
Clickjacking	Frame Options Restriction	X-Frame-Options: SAMEORIGIN prevents unauthorized domain embedding	SecurityHeadersMiddleware.php
Malicious File Upload / Shells	Deep Binary Media Inspection	getimagesize() binary check, MIME whitelist, and random file renaming	app/shared/auth_actions.php
Ballot Stuffing / Race Voting	Database Concurrency Locking	InnoDB SELECT ... FOR UPDATE locks within transactions ensure single vote	app/shared/election_actions.php
Attendance Spoofing	QR Token Verification	Event & student identifier resolution, duplicate scan denial, method logging	app/shared/attendance_actions.php
Unauthorized DB Reset	Installer Gating (install.lock)	Lockfile rejection blocks unauthorized execution of setup.php	app/shared/setup.php
Accountability Gaps	Centralized Audit Logging	Structured capture of user ID, IP address, action, and target in audit_logs	notification_actions.php